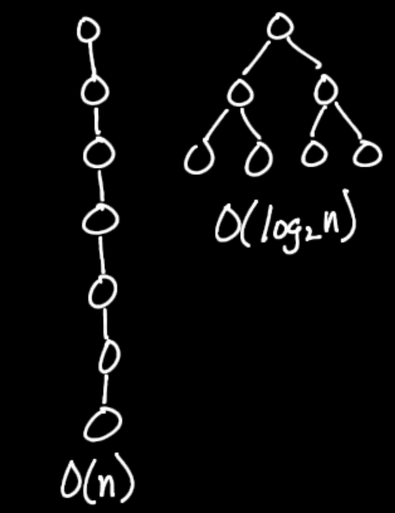
① BST efficiency depends on the height

Worst case: O(n)

best case: O(log₂n)



height is sensitive to the order of inserts, deletes and key values

The more "linear" the tree, the more "unbalanced" it is

Solution - balanced tree

Common trees
- 2-3 tree
- 2-3-4 tree
- B-tree
- Red-Black tree
- AVL tree

→

② 2-3 Tree

Nodes
- 1 value, exactly 2 children
- 2 values, exactly 3 children
- except leafs

Values are ordered in a node

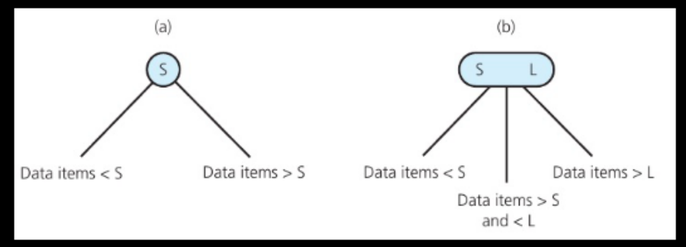
Technically not a BST

Maintains the BST property

Inorder traversal
- l - S - m - L - right

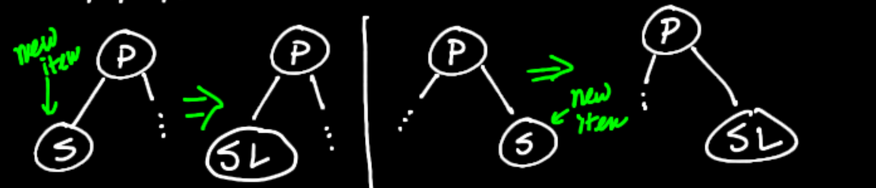
Never taller than a min height binary tree

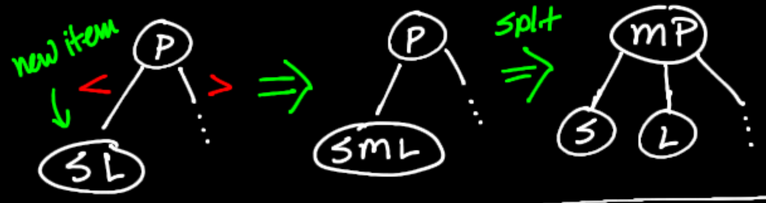
Balancing requires split, redistribution, and merges

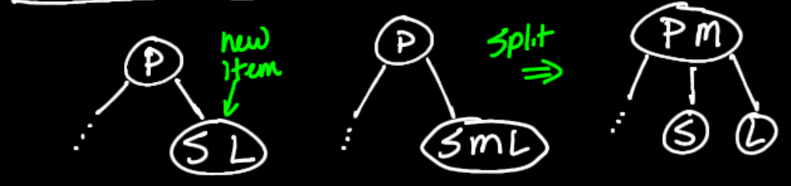


③ Leaf

P, S, M, L are relative ordering

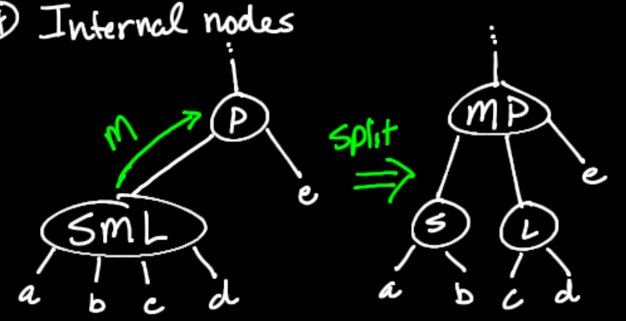


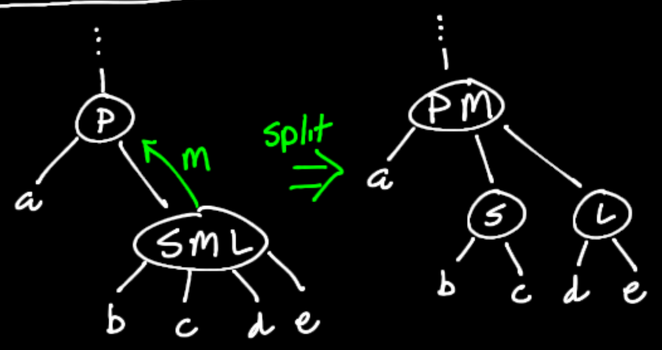




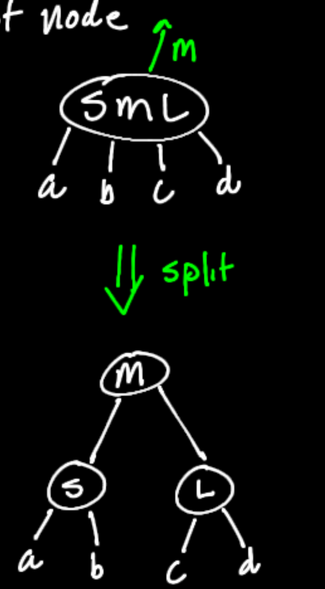
→

④ Internal nodes



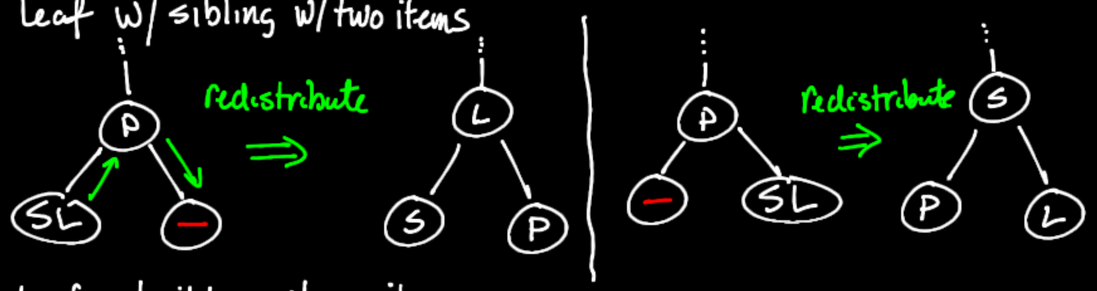


Root node

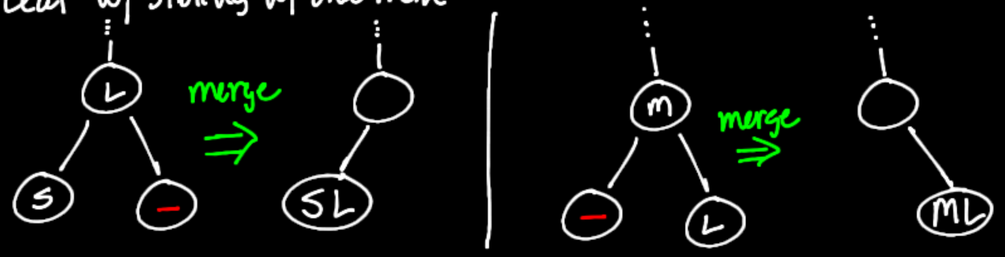


What about deleting?

⑤ Leaf w/ sibling w/ two items

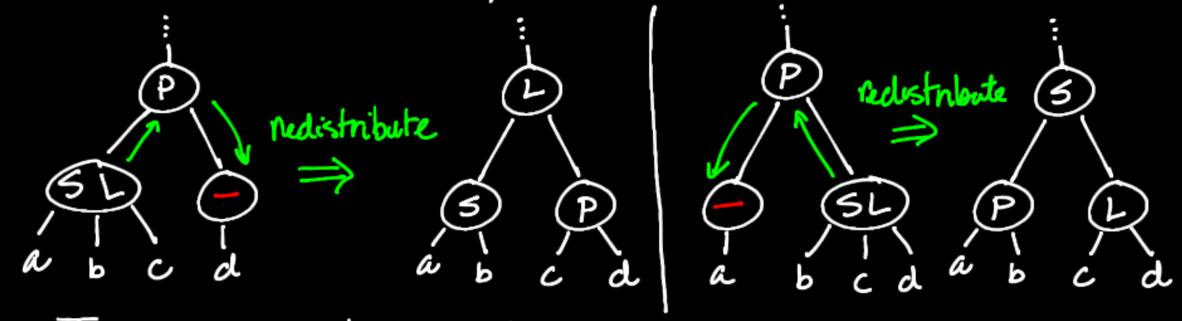


Leaf w/ sibling w/ one item

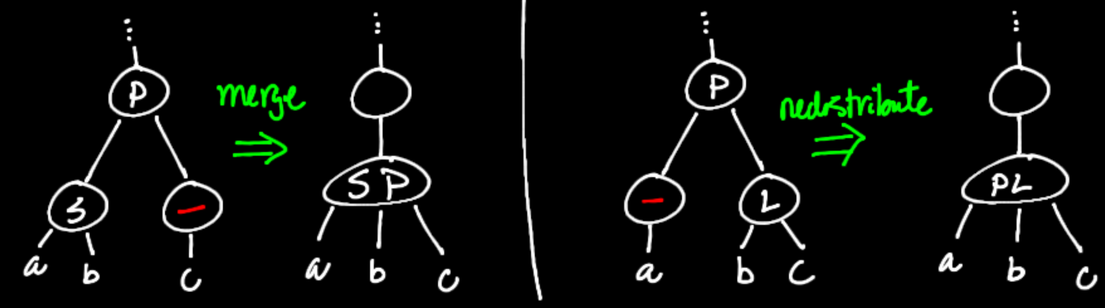


Now you have to solve the empty node problem at the level above it

⑥ Internal node w/ sibling w/ two items

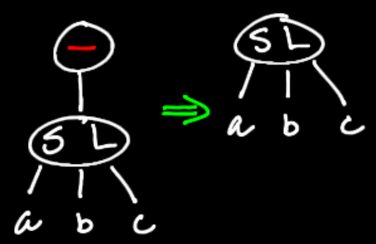


Internal node w/ sibling w/ one item

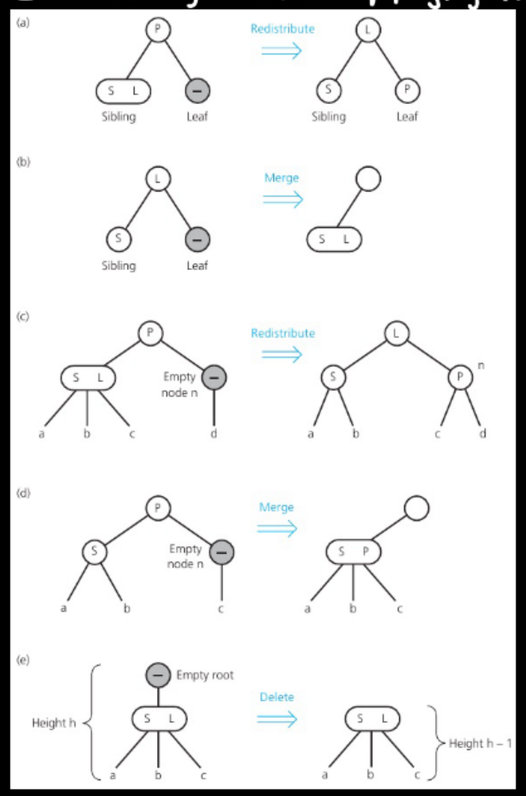


Now we solve the problem at the level above

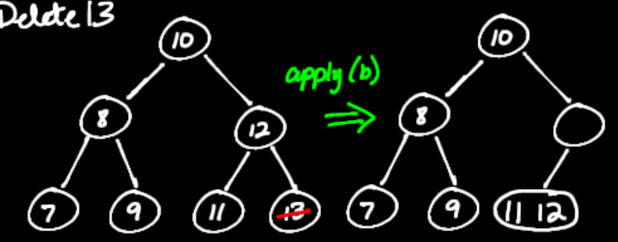
⑦ Root

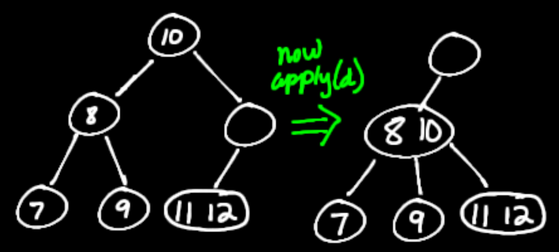


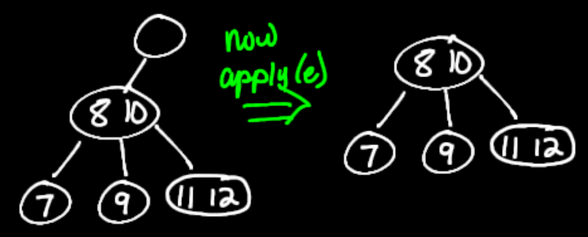
⑧ Deleting involves applying the rules as the issue moves up the tree



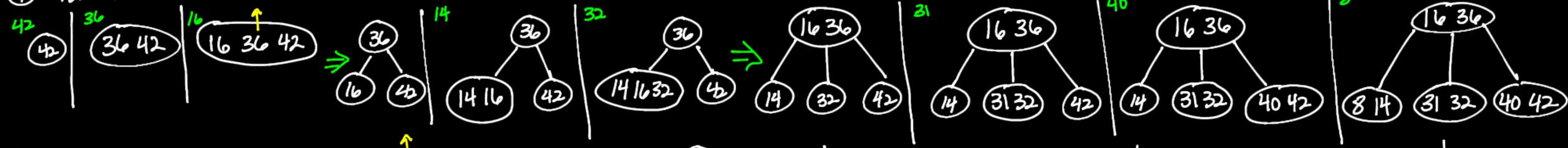
Delete 13

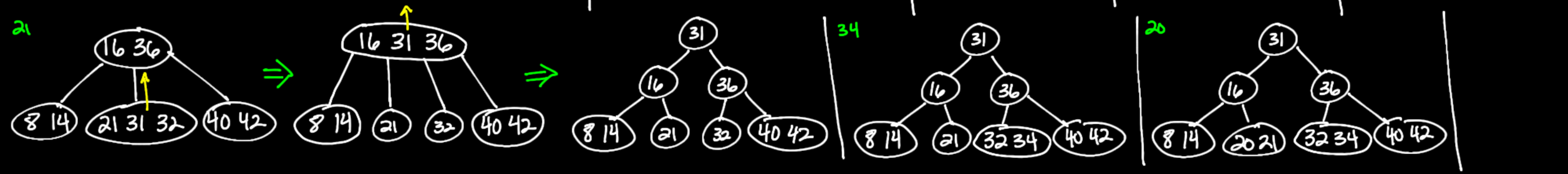


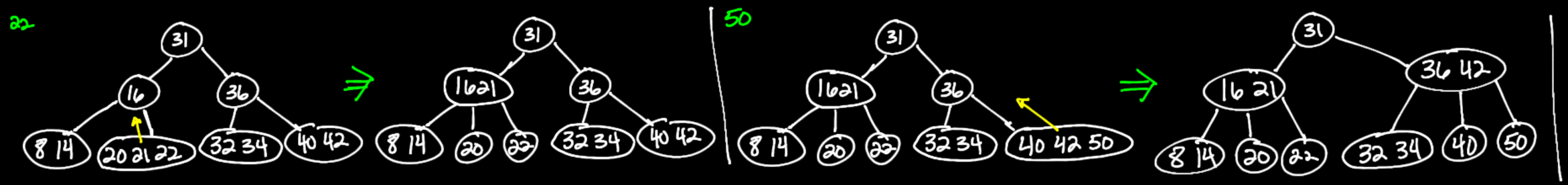


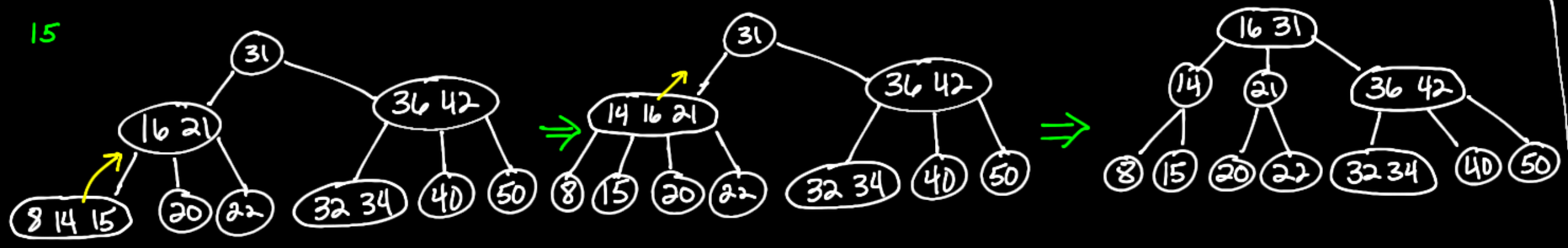


⑨ 42 36 16 14 32 31 40 8 21 34 20 22 50 15

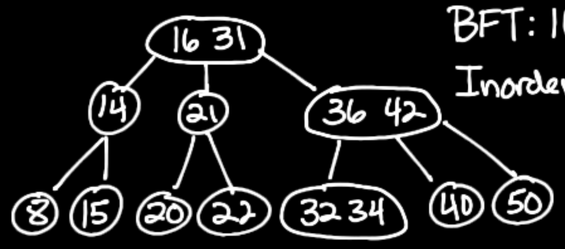








challenge:
insert 33
delete 16
shuffle the numbers and insert
shuffle the numbers and delete



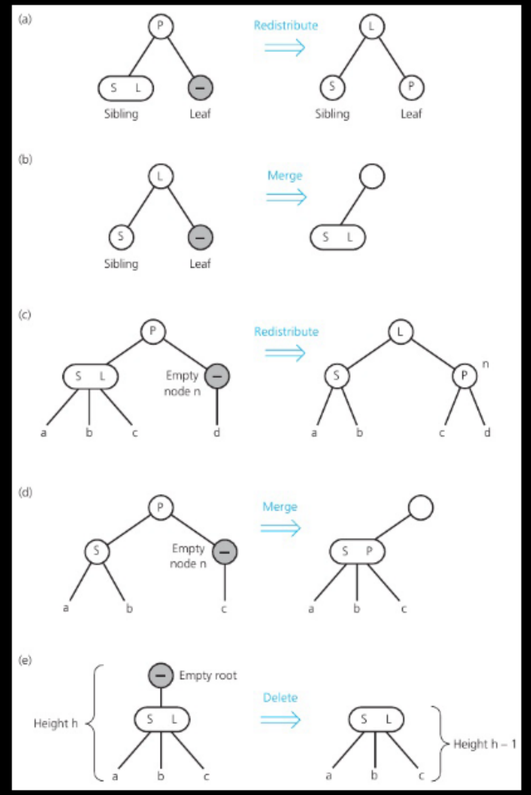
BFT: 16 31 14 21 36 42 8 15 20 22 32 34 40 50

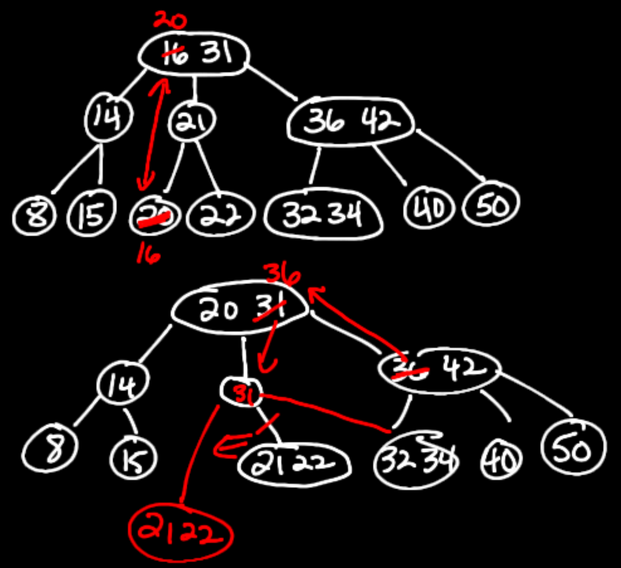
Inorder : left, visit, right

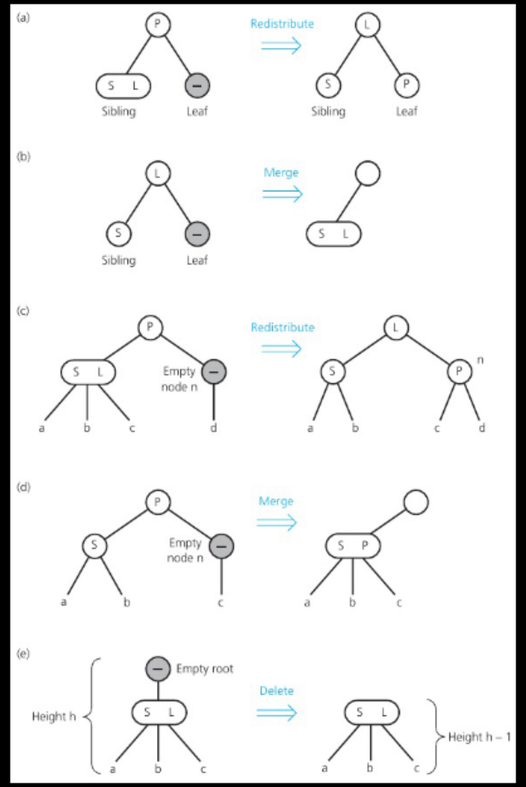
left, visit S, mid, visit L, right

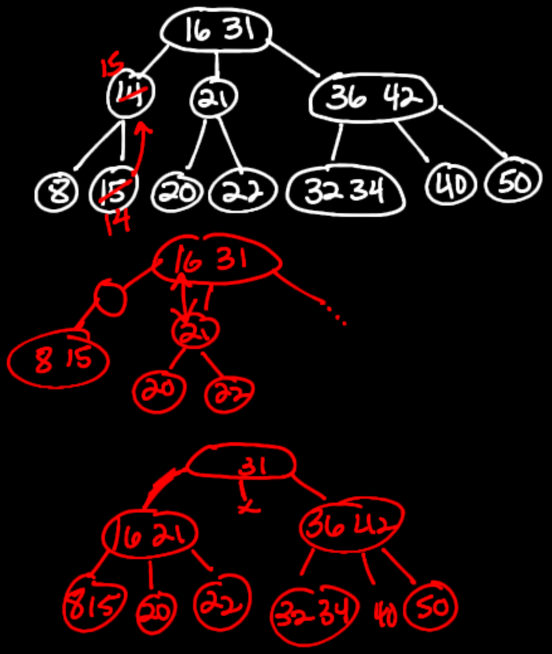
↓

⑩ Deleting


(a)
Redistribute
Sibling
Leaf
(b)
Merge
(c)
Redistribute
Empty node n
(d)
Merge
(e)
Empty root
Height h
Delete
Height h – 1


20
16 31
14
21
36 42
8
15
20
22
32 34
40
50
16
36
20 31
14
31
36 42
8
15
21 22
32 34
40
50
21 22
20 36
14
31
42
8
15
21 22
32 34
40
50


(a)
Redistribute
Sibling
Leaf
(b)
Merge
(c)
Redistribute
Empty node n
(d)
Merge
(e)
Empty root
Height h
Delete
Height h – 1


16 31
15
14
21
36 42
8
15
14
20
22
32 34
40
50
16 31
8 15
20
22
31
16 21
36 42
8 15
20
22
32 34
40
50